IC Lab Formal Verification
Bonus Quick Test
2025 Fall

Name: 郭家瑜 **Student ID:** 314591015 **Account:** iclab041

(a) What is Formal verification?

Formal verification is a mathematical, exhaustive verification methodology that checks whether a design satisfies a set of properties using formal engines, rather than relying on manually written tests. Formal tools systematically explore all reachable states and all legal input combinations, enabling complete proof of correctness or generation of counterexamples when properties are violated.

What's the difference between Formal and Pattern based verification? And list the pros and cons for each.

Aspect	Formal Verification	Pattern-Based (Simulation) Verification
Stimulus Generation	Automatically explores all possible legal stimuli and state space (breadth-first).	Depends on user-written test patterns or constrained-random stimuli; cannot cover all states.
Bug-Finding Strategy	Exhaustive and mathematical (proof-based).	Depth-first search following stimulus sequences.
Result Type	Provides proof, bounded proof, counterexample, or unreachable.	Provides waveform results only for tested scenarios.
Coverage Characteristics	Can prove correctness or discover unreachable logic; sees all corner cases.	Limited by creativity and completeness of testbench; may miss rare scenarios.

Formal Verification

Formal verification uses mathematical techniques to exhaustively analyze all possible states and input scenarios of a design. It aims to ensure correctness by thoroughly checking every potential case, including corner cases.

Pros:

- Exhaustive exploration → finds corner cases simulation will never reach.
- Detects unreachable logic, overconstraints, dead code (coverage analysis).
- High confidence due to mathematical proof.
- Requires smaller testbench (assertions + assumptions only).

Cons:

- Computationally expensive for large, highly sequential designs.
- Requires carefully written constraints and properties.
- State-space explosion limits scalability.

Pattern-Based Verification (Simulation)
Pattern-based verification relies on generating specific input patterns and comparing the design's outputs to pre-determined golden results. It is primarily simulation-based and tests a subset of possible scenarios.

Pros:
- Fast and efficient for large data-path–heavy or computation-intensive modules.
- Intuitive debugging through long waveform traces.
- Flexible—designer can target specific complex scenarios.

Cons:
- Limited coverage; cannot guarantee correctness.
- May miss deep corner cases or rare concurrency bugs.
- Quality depends heavily on the designer's creativity and pattern completeness.

Summary:
Formal ≈ exhaustive and mathematical.
Pattern-based ≈ practical but inherently incomplete.

(b) What is glue logic?

Glue logic is auxiliary RTL written purely to help describe complex properties. Examples include internal flags, counters, or state trackers used to capture abstract concepts such as "in_packet," "transaction_started," or "no_gap_between_SOP_and_EOP."

Why will we use glue logic to simplify our SVA expression?

1. Simplifies complex temporal properties:
   Instead of encoding long SVA sequences repeatedly, glue logic expresses the concept once, and assertions refer to the simplified signal.

2. Improves readability and maintainability:
   Clean SVA → easier for humans to understand and debug.

3. Reduces SVA complexity:
   Glue logic avoids deep nesting of temporal operators (##, until_with, throughout, etc.).

4. No cost in Formal:
   The JasperGold engine treats SVA-only and SVA+glue-logic approaches equivalently—no performance penalty.

5. Avoids duplication and errors:
   Reusing glue logic reduces the risk of subtle bugs from duplicated SVA code.

Summary:
Glue logic transforms complicated conditions into intuitive signals, making assertions shorter, more accurate, and easier to verify.

(c) What is the difference between Functional coverage and Code coverage?

Functional Coverage

- User-defined (covergroups, SVA cover properties).

- Measures whether intended behaviors, sequences, or scenarios have been exercised.
- Represents the verification plan and checks whether design functionality is validated.
- Accurate but requires manual planning and writing.

Code Coverage

- Automatically generated (statement, branch, expression coverage).
- Measures which HDL structures were executed.
- Ensures structural completeness but does not guarantee correctness.
- May include noise or meaningless paths.

What's the meaning of 100% code coverage, could we claim that our assertion is well enough for verification? Why?

No, achieving 100% code coverage does not necessarily mean that the assertions or tests are sufficient for thorough verification.

Reasons:

1. Execution does not imply correctness:
   A line executing once does not prove the behavior is correct.
2. Functional intent may remain untested:
   Even with full structural coverage, many functional scenarios may never be checked.
3. Assertions may be missing, incomplete, or incorrect:
   Code coverage cannot detect assertion quality.
4. False sense of completeness:
   Code might execute under trivial or unmeaningful scenarios (e.g., value is X or forced).

   Thus, 100% code coverage ≠ fully verified design.
   Functional coverage + checker coverage (assertions) + proof coverage are required for real completeness.

(d) What is the difference between COI coverage and proof coverage for realizing checker's completeness? Try to explain from the meaning, relationship, and tool effort perspective.

1. Meaning

COI Coverage (Cone-of-Influence Coverage)

- Identifies all design logic that can potentially affect an assertion's result.
- Purely structural analysis (no formal engine required).
- Items outside the COI indicate holes in the assertion set.

Proof Coverage

- Identifies which parts of the COI were actually verified via full or bounded proofs.

- Represents the “proof core”—the logic actively contributing to the property’s correctness.
- Requires formal engines and multiple proof iterations.

2. Relationship

- Proof coverage ⊆ COI coverage
  COI = maximum possible influence.
  Proof = actually exercised & proven portion.

3. Tool Effort

Metric	COI Coverage	Proof Coverage
Engine use	No	Yes
Speed	Very fast	Slow (requires proofs)
Accuracy	Structural → may overestimate influence	Functional → precise proof-based checking
Purpose	Find missing assertions	Find unverified reachable logic

Summary:
- COI tells you whether your assertion touches the logic.
- Proof coverage tells you whether your assertion actually verifies the logic.
- Both are needed for checker completeness.

(e) What are the roles of ABVIP and scoreboard separately?
Try to explain the definition, objective, and the benefit.

ABVIP (Assertion-Based Verification IP)

Definition:
A library of protocol checkers (e.g., AXI, AHB) consisting of pre-written assertions and monitors that validate protocol compliance.

Objective:
- Ensure the DUT strictly follows protocol timing rules, ordering rules, handshakes, and transaction semantics.
- Reduce designer workload in writing complex protocol assertions.

Benefits:
- Highly reusable and standardized.
- Saves significant time and reduces human error.
- Ensures robust, vendor-proven checking quality.

Scoreboard

Definition:
A formal or simulation verification component that tracks input-output transactions and checks correctness, ordering, duplication, corruption, and drop behaviors.

Objective:

- Ensure functional correctness of data-path behavior.
- Compare DUT outputs with expected or reconstructed values.
- Detect violations such as dropped packets, reordering (unless allowed), or corruption.

Benefits:

- Automatically verifies large data sequences.
- Helps catch deep functional bugs unrelated to protocol timing.
- Works well for FIFOs, routers, memory subsystems, and bus bridges.

Summary:

- ABVIP → protocol correctness.
- Scoreboard → data correctness.
  Together, they form a complete formal verification ecosystem.